

UNIT-III

What is a Decision Tree in Machine Learning?

A **Decision Tree** is a supervised learning algorithm used for both **classification** and **regression** tasks. It works by dividing the data into subsets based on certain features, which helps in predicting the target value (class labels in classification).

In classification, the Decision Tree algorithm predicts categorical labels (e. g., "Yes" or "No"). The tree structure consists of:

- **Nodes:** Each node represents a decision based on one of the features.
- **Edges:** These represent the outcome of the decision (e. g., "Yes" or "No").
- **Leaves:** These represent the predicted class labels.

Key Terminology:

- **Root Node:** The top most node that represents the first decision.
- **Splitting:** Dividing the dataset into subsets based on a feature.
- **Pruning:** Reducing the size of the tree to avoid over fitting.
- **Leaf Node:** The final node that provides the classification label.
- **Branch:** The decision that leads to further splits or leaves.

How Does a Decision Tree Work?

The goal of a Decision Tree is to classify data by recursively splitting the dataset at each internal node, based on the feature that best divides the data. The process continues until a stopping criterion is met, such as all data in a node belonging to a single class or the tree reaching a maximum depth.

The splitting is determined by a criterion like:

- **Gini Impurity:** Measures the "impurity" of a node. The goal is to minimize it at each split.
- **Entropy (Information Gain):** Measures the "uncertainty" of the dataset. The feature that provides the maximum information gain is selected for splitting.

Key Steps for Building a Decision Tree:

1. **Select the Best Feature:** Choose the feature that best splits the data (e. g., based on Gini Impurity or Information Gain).
2. **Split the Data:** Divide the data into subsets based on the chosen feature.
3. **Repeat:** Recursively split the data on the remaining features until a stopping condition is reached.
4. **Make Predictions:** Once the tree is built, predictions are made by following the path from the root to the leaf node corresponding to the input data.

Simple Example: "Play Tennis" Dataset

Let's go through a **simple classification example** where we build a Decision Tree to predict whether to play tennis or not based on weather conditions. Here's the dataset:

Outlook Temperature Humidity Wind Play Tennis

Sunny	Hot	High	Weak	No
Sunny	Hot	High	Strong	No
Overcast	Hot	High	Weak	Yes
Rain	Mild	High	Weak	Yes
Rain	Cool	Normal	Weak	Yes
Rain	Cool	Normal	Strong	No
Overcast	Cool	Normal	Strong	Yes
Sunny	Mild	High	Weak	No
Sunny	Cool	Normal	Weak	Yes
Rain	Mild	Normal	Weak	Yes

Steps to Build the Decision Tree:

- 1. Choose the Best Feature to Split On:** We start by calculating the **Gini Impurity** or **Entropy** of each feature to find the one that best separates the data.
- 2. Splitting the Data: Let's assume after calculation, the Outlook feature is chosen as the best feature to split on.**
- 3. Create Nodes and Branches:**
 - If Outlook=Sunny, we check the Humidity feature.
 - If Outlook=Overcast, the prediction is Yes (since all instances are "Yes").
 - If Outlook=Rain, we check the Wind feature.
- 4. Recursive Splitting:**
 - For Sunny, if Humidity=High, the prediction is No.
 - For **Rain**, if **Wind = Weak**, the prediction is **Yes**, and if **Wind = Strong**, the prediction is **No**.

Final Tree:

The Decision Tree looks like this:

```

      Outlook
     /  |  \
    /   |   \
Sunny Overcast Rain
 /  \  /  \
Humidity Wind Wind
 /  \  /  \  \
High Normal Weak Strong Weak Strong
| | | | |
No Yes Yes No Yes No

```

Code Example in Python (using scikit-learn):

Here's how to implement this in Python using the scikit-learn library.

```

import pandas as pd
from sklearn. Tree import Decision Tree Classifier
from sklearn import tree

#Define the dataset data = {
    'Outlook':['Sunny','Sunny','Overcast','Rain','Rain','Rain','Overcast','Sunny', 'Sunny',
    'Rain'],
    'Temperature':['Hot','Hot','Hot','Mild','Cool','Cool','Cool','Mild','Cool','Mild'],
    'Humidity': ['High', 'High', 'High', 'High', 'Normal', 'Normal', 'Normal', 'High', 'Normal',
    'Normal'],
    'Wind':['Weak','Strong','Weak','Weak','Weak','Strong','Strong','Weak','Weak', 'Weak'],
    'Play Tennis':['No','No','Yes','Yes','Yes','No','Yes','No','Yes','Yes']
}

# Convert to Data Frame
df=pd. Data Frame
(data)

#Encode categorical variables as numeric values
df_encoded = pd. get_dummies (df. drop ('Play Tennis', axis=1))
df_encoded ['Play Tennis'] =df ['Play Tennis']. map ({'No': 0,'Yes': 1})

#Features (X) and target (y)
X = df_encoded
y=df_encoded ['Play Tennis']

#Create Decision Tree Classifier
clf = Decision Tree Classifier (criterion='gini', max_depth=3)# Limit tree depth to avoid
overfitting
clf=clf. fit (X, y)

#Visualize the decision tree
tree. plot_tree (clf, feature_names=X. columns, class_names=['No','Yes'], filled=True)

```

Explanation of Code:

1. **Data Preparation:** The dataset is stored in a dictionary and converted into a Pandas Data Frame. We use `pd. get_dummies ()` to convert categorical features (Outlook, Temperature, etc.) into numerical values.
2. **Encoding Target Variable:** The target variable "Play Tennis" is encoded as 0 (No) and 1 (Yes) using `. map ()`.
3. **Decision Tree Classifier:** We create a Decision Tree Classifier and set the criterion to "gini"(for Gini Impurity). We also limit the maximum depth of the tree to 3 to avoid overfitting.
4. **Training the Model:** The classifier is trained using the `fit ()` method with the feature matrix X and target vector y.

5. Tree Visualization: The tree is visualized using `tree. plot_tree ()`, where `feature_names` are the column names of `X`, and `class_names` are the labels for "No" and "Yes."

Measures and Properties used in Decision Trees for Classification

1. Measures Used to Split the Data in Decision Trees

The main goal of a Decision Tree is to create branches and leaf nodes in a way that maximizes the homogeneity of the target class within each leaf. To do this, **measures** like **Gini Impurity** and **Entropy (Information Gain)** are used to evaluate how well a feature splits the data.

Gini Impurity

Gini Impurity measures the degree of impurity in a node. It's a way of quantifying how mixed the classes are within a node. A lower Gini value indicates that the node is purer (I. e., the class is more homogeneous).

2 Entropy (Information Gain)

Entropy measures the uncertainty in a dataset. A node with **high entropy** has more uncertainty, meaning the classes are mixed. A node with **low entropy** is more certain, meaning the data points in that node belong to the same class.

2. Properties of Decision Trees

The **properties** of Decision Trees that influence their behavior and performance include **overfitting**, **depth of the tree**, and **interpretability**. These properties are important when constructing a Decision Tree to ensure it generalizes well and avoids errors.

Overfitting

One of the major issues with Decision Trees is **overfitting**. This happens when the tree becomes too complex (too deep) and starts to memorize the training data rather than learning general patterns. This results in poor performance on new, unseen data.

Overfitting occurs when:

- The tree grows too deep and becomes highly specific to the training data.
- The model performs well on the training set but fails to generalize to the test set.

To prevent overfitting, techniques like pruning, setting a maximum depth, or using **Ensemble methods (e. g., Random Forests)** are applied.

Maximum Depth

The **maximum depth** of the tree defines how many levels the tree can have. Limiting the depth helps control overfitting by ensuring that the tree doesn't get too complex. For example:

- A shallow tree may under fit the data, failing to capture important patterns.
- A deep tree may over fit the data, capturing noise rather than useful patterns.

Interpretability

One of the key strengths of Decision Trees is their interpretability. The tree is visual and can be easily understood by humans, making it easy to see how decisions are made. For example, the decision path from the root to a leaf is straightforward to interpret:

- —If Outlook=Sunny and Humidity=High, then Predict No.||

This makes Decision Trees an attractive model for applications where understanding the decision-making process is important.

3. Simple Example to Demonstrate Measures and Properties

Let's walk through an example where we apply the Gini Impurity measure to a small dataset and build a simple decision tree.

Dataset: "Play Tennis" based on weather conditions

Outlook Temperature Humidity Wind Play Tennis

Sunny	Hot	High	Weak	No
Sunny	Hot	High	Strong	No
Overcast	Hot	High	Weak	Yes
Rain	Mild	High	Weak	Yes
Rain	Cool	Normal	Weak	Yes
Rain	Cool	Normal	Strong	No
Overcast	Cool	Normal	Strong	Yes
Sunny	Mild	High	Weak	No
Sunny	Cool	Normal	Weak	Yes
Rain	Mild	Normal	Weak	Yes

Step 1: Calculate Gini Impurity for Each Feature

Let's first calculate the Gini Impurity for the feature "Outlook."

- For Outlook=Sunny, the data points are:
 - "No"(3 instances) and "Yes"(0 instances).
 - The Gini Impurity for Sunny = $1 - (3/3)^2 = 1 - (3/3)^2 = 0$ (pure, since all "No").
- For Outlook=Overcast, the data points are:
 - "Yes"(2 instances) and "No"(0 instances).
 - The Gini Impurity for Overcast = $1 - (2/2)^2 = 1 - (2/2)^2 = 0$ (pure, since all "Yes").
- For Outlook=Rain, the data points are:
 - "Yes"(3 instances) and "No"(1 instance).
 - The Gini Impurity for Rain = $1 - ((3/4)^2 + (1/4)^2) = 1 - (9/16 + 1/16) = 1 - 10/16 = 0.375$

Step 2: Select the Best Feature to Split on

Looking at the Gini Impurity:

- Sunny: Gini=0 (pure).**
- Overcast: Gini=0 (pure).**
- Rain: Gini= 0.375.**

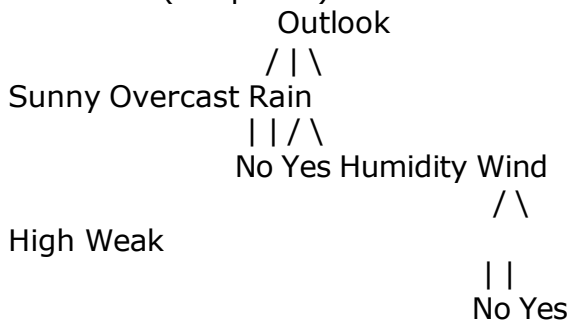
Since Outlook = Sunny and Outlook = Overcast give the purest splits (Gini = 0), either of these would be the best split. But since both splits lead to pure leaves, Outlook is the best feature to split on.

Step 3: Build the Tree

We create the decision tree by first splitting on **Outlook**:

- If Outlook=Sunny, predict "No"(pure class).
- If Outlook=Overcast, predict "Yes"(pure class).
- If Outlook=Rain, further split based on Humidity or Wind.

Final Tree (Simplified):

**Decision Trees for Regression**

Decision Trees can also be used for **regression** tasks, where the goal is to predict continuous values rather than categorical labels (as in classification). The structure of the Decision Tree for regression is similar to that for classification, but instead of predicting class labels, it predicts a continuous value.

Key Concepts

- **Regression Tree:** A Decision Tree used for predicting continuous values. The main idea is to split the data into subsets where each subset has a homogeneous outcome (or at least closer values).
- **Splitting Criteria:** Instead of Gini Impurity or Entropy (which are used in classification), for regression, the commonly used criterion is to minimize the **Mean Squared Error (MSE)** between the true and predicted values. The goal is to choose splits that reduce the variance of the target variable within each resulting subset.
- **Prediction:** In a regression tree, each **leaf node** will contain the **mean** (or sometimes median) value of the target variable for the data points in that leaf. When making a prediction, the tree will follow the splits to the corresponding leaf and return the mean value of that leaf.

Steps in Regression Tree:**1. Choose the Best Feature to Split On:**

- The decision tree algorithm evaluates different features and possible splits by calculating how much the variance in the target variable reduces after the split.
- The splitting criteria used in regression trees is Variance Reduction or **Mean Squared Error (MSE)**.

2. Split the Data:

- The feature and threshold that minimize the MSE or variance are chosen for the split.

3. Repeat the Process:

- This process is repeated recursively, splitting the data at each node, until a stopping criterion is met (such as maximum tree depth, minimum samples at a node, or a predefined level of variance reduction).

4. Prediction:

- When making a prediction, you follow the tree's structure and arrive at a leaf node. The value at that leaf node is the predicted value (the mean value of the target variable for the data points in that leaf).

Simple Example: Predicting House Prices

Let's consider an example where we have a dataset with features about houses (like the number of bedrooms and the size of the house) and we want to predict the **price of the house**.

Example Dataset: House Prices**Bedrooms Size (sqft) Price (\$)**

2	800	150, 000
3	1200	200, 000
3	1100	180, 000
4	1600	250, 000
4	1700	280, 000
5	2200	350, 000
5	2100	330, 000

Step-by-Step Process:**1. Choose the Best Feature to Split On (Variance Reduction):**

- We calculate the variance of the target variable (Price) in the entire dataset and for potential splits based on features like Bedrooms and Size.
- For example, we might start by considering if splitting based on Bedrooms or Size reduces the variance in the target (Price).

2. First Split (Based on Size): Let's say that when we split based on Size, the data splits into two groups:

- **Group 1 (Size ≤ 1500 sqft): Houses with smaller sizes (e. g., 800, 1200, 1100 sqft).**
- **Group 2 (Size > 1500 sqft): Larger houses (e. g., 1600, 1700, 2200 sqft).**

For Group 1 (houses with Size ≤ 1500 sqft):

- Mean price = $(150,000 + 200,000 + 180,000) / 3 = 176,666$

For Group 2 (houses with Size>1500 sqft):

- Mean price=(250, 000+280, 000+350, 000+330, 000)/4=302, 500

3. Second Split (Based on Bedrooms): We now recursively split the groups further. For example, in Group 1 (Size≤1500 sqft), we might decide to split based on the number of Bedrooms:

- Group 1.1 (Bedrooms=2 or 3): Houses with 2 or 3 bedrooms (e. g., 800, 1200, 1100 sqft).**
- Group 1.2 (Bedrooms=4): Houses with 4 bedrooms (e. g., 1600 sqft).**

For Group 1.1 (Bedrooms=2 or 3):

- Mean price=(150, 000+200, 000+180, 000)/3=176, 666

For Group 1.2 (Bedrooms=4):

- Mean price=250, 000 (only one data point, no variance)

Similarly, for Group 2 (Size>1500 sqft), we might split based on Bedrooms and get further subgroups.

- 4. Repeat Until Stopping Criteria Met: This splitting continues recursively, and at some point, the tree stops splitting (when it reaches a predefined maximum depth or when further splitting doesn't reduce the variance much).**
- 5. Prediction: Once the tree is built, to predict the price of a new house, the decision tree follows the appropriate splits based on the new houses features (e. g., number of bedrooms, size) and finally arrives at a leaf. The predicted price is simply the mean price of the data points in that leaf.**

Visualize the tree plt. figure

(figsize=(12, 8))

plot_tree (regressor, feature_names=['Bedrooms','Size'], filled=True, rounded=True, fontsize=10)

plt. show ()

Explanation:

- 1. Data Preparation: We create a Data Frame from the example dataset with features Bedrooms and Size, and target variable Price.**
- 2. Train-Test Split: We split the data into training and testing sets (70%training, 30% testing).**
- 3. Train Decision Tree: We create a Decision Tree Regressor object, fit the model on the training data, and make predictions.**
- 4. Tree Visualization: We visualize the regression tree to understand how splits are made and the predicted values at the leaves.**

Output (example):

- The predicted prices for the test data are printed.

- **The visualized decision tree would show how the data is split based on Bedrooms And Size.**

What is Bias-Variance Trade-off?

In machine learning, the Bias-Variance Trade-off refers to the balance between two sources of error that affect the performance of predictive models:

1. **Bias: The error that is introduced by the model's assumptions. A model with high bias oversimplifies the data and leads to underfitting (not capturing the complexity of the data).**
2. **Variance: The error that is introduced by the model's sensitivity to the specific training data. A model with high variance tries to fit every little detail of the training data and leads to overfitting (capturing noise and fluctuations that aren't useful for generalization).**

As we adjust the complexity of a model, bias and variance change in opposite directions. The challenge is to find a balance where the total error is minimized.

Bias-Variance Trade-off: Simple Example

Imagine we're trying to model the relationship between the number of hours studied and exam score. Our goal is to predict the exam score based on the number of hours a student studies.

Data:

- Hours studied (X): [1, 2, 3, 4, 5]
- Exam score (Y): [2, 4, 6, 8, 10]

This data has a simple linear relationship. The true function that describes the data is:

True function: $Y = 2X$

Now, let's analyze what happens with different types of models (with varying complexity).

1. High Bias (Underfitting)

A simple model with high bias might be a **linear regression** with a fixed relationship between hours and scores, such as:

$Y = 3X$

In this case, the model doesn't capture the true relationship ($Y = 2X$). The error is large because the model assumes the slope is 3 instead of 2.

- **Bias is high because the model is too simple (we assumed a wrong relationship).**
- **Variance** is low because the model doesn't change much with different datasets.
- **Result: The model consistently predicts poorly and has a large bias error.**

2. High Variance (Overfitting)

Now, imagine we use a high-degree polynomial regression to fit the data. For instance, we might use a polynomial that fits the data perfectly, like:

$Y = X^3 - 3X^2 + 2X$

This model would create a curve that fits the five points exactly, with a very low error on the training data.

- **Bias** is low because the model is very flexible and can match the data closely.
- **Variance is high because the model fits the training data perfectly, but it may perform poorly on new, unseen data (overfitting).**

- Result: The model captures noise in the training data, leading to over fitting and high variance error when tested on new data.

3. Balanced Bias-Variance

To achieve the best model, we need to find the sweet spot where both bias and variance are balanced. This usually occurs with a moderately complex model.

For example, a linear regression model like:

$$Y = 2X$$

Is a good fit for this problem? It accurately captures the true relationship without being too simple or too complex.

- **Bias** is low because the model correctly reflects the true relationship between hours studied and exam score.
- **Variance** is also low because the model is not over fitting or under fitting.
- **Result: The model generalizes well to new data, with minimal bias and variance.**

Visualizing the Bias-Variance Trade-off

To summarize, here's a conceptual break down of how bias and variance behave with Increasing model complexity:

- **Low complexity models (high bias, low variance):** The model is too simple, leading to under fitting and poor performance. Bias is high, and variance is low.
- **High complexity models (low bias, high variance):** The model is too complex, overfitting the data and capturing noise. Bias is low, and variance is high.
- **Optimal complexity (balanced bias and variance):** The model is just complex enough to capture the patterns in the data without overfitting or under fitting. This is the sweet spot.

Key Points:

- **Bias:** Errors due to overly simplistic models (under fitting).
- **Variance:** Errors due to overly complex models (overfitting).
- **Total Error:** The sum of bias squared, variance, and irreducible error (inherent noise in the data). The goal is to minimize total error.

Random Forests for Classification and Regression

Random Forests are a type of ensemble learning method used for both classification and regression tasks. They are based on the idea of combining multiple decision trees to improve the overall model performance. Random Forests are widely used because they help overcome the limitations of individual decision trees, such as overfitting.

Key Concepts:

- **Ensemble Learning:** Combining predictions from multiple models to improve accuracy and robustness.
- **Bagging (Bootstrap Aggregating):** Random Forests use bagging, a technique where multiple datasets are generated by randomly sampling the training data

-

Wither placement (bootstrapping), and each data set is used to train a separate decision tree.

- **Random Feature Selection:** At each split in the tree, instead of considering all features, a random subset of features is selected. This helps in creating diverse trees and reduces correlation among the individual trees in the forest.

The predictions from all the individual trees are aggregated to form the final prediction:

- For Classification, the most common class predicted by the trees is taken as the final output (majority voting).
- For Regression, the average of the predictions from all the trees is taken as the final output.

Random Forests for Classification:

In classification, Random Forests are used to classify data into discrete categories (e. g., predicting whether a customer will buy a product: Yes or No).

Process for Classification:

1. **Create Bootstrapped Datasets:** Randomly sample data points with replacement to create multiple training sets.
2. **Train Decision Trees:** Train a separate decision tree on each bootstrapped dataset.
3. **Make Predictions:** When making predictions for new data, each tree in the forest votes for a class label.
4. **Majority Voting:** The class that receives the most votes from the trees is selected as the final prediction.

Example: Predicting Whether a Customer Will Buy a Product (Yes/No)

Let's say we have a data set where the task is to predict whether a customer will buy a product based on features like age, income, and previous purchase behavior.

Age Income Previous Purchase Buy Product (Target)

25	30 k	No	No
30	50 k	Yes	Yes
35	60 k	No	No
40	80 k	Yes	Yes
45	70 k	Yes	Yes
50	90 k	No	No

Steps:

1. **Create Bootstrapped Datasets:** Randomly sample rows from the data set to create multiple training datasets.
2. **Train Decision Trees:** Each tree is trained on a different dataset, with random feature selection at each split.
3. **Make Predictions:** For a new customer, each tree in the forest predicts whether they will buy the product or not.

4. **Majority Voting:** The final prediction is based on the majority vote among the trees.

Random Forests for Regression:

In regression, Random Forests predict continuous values (e. g., predicting house prices based on features like size, number of rooms, and location).

Process for Regression:

1. **Create Bootstrapped Datasets:** Randomly sample data points with replacement to create multiple training sets.
2. **Train Decision Trees:** Train a separate decision tree on each bootstrapped dataset.
3. **Make Predictions:** For new data, each tree in the forest predicts a continuous value.
4. **Average Predictions:** The final prediction is the average of the predictions from all the trees in the forest.

Example: Predicting House Prices

Let's say we have a dataset of houses, and the task is to predict the price of a house based on features like size, number of bedrooms, and location.

Size (sqft) Bedrooms Location Price (Target)

800	2	Suburban	150, 000
1200	3	Urban	250, 000
1500	3	Urban	350, 000
1800	4	Suburban	400, 000
2200	4	Urban	500, 000
2000	3	Rural	300, 000

Steps:

1. **Create Bootstrapped Datasets:** Randomly sample rows from the data set to create multiple training datasets.
2. **Train Decision Trees:** Each tree is trained on a different data set, and random features are chosen at each split.
3. **Make Predictions:** For a new house, each tree in the forest predicts the price.
4. **Average Predictions:** The final prediction is the average of all the trees' predictions.

Advantages of Random Forests:

1. **Robust to Overfitting:** By averaging predictions from multiple-trees and randomly selecting features, Random Forests reduce the risk of overfitting compared to a single decision tree.
2. **Handles Missing Data:** Random Forests can handle missing data by using surrogate splits or simply using available data from other

3. **Works Well with Large Datasets:** Random Forests can handle large data sets with high dimensionality (many features).
4. **Feature Importance:** Random Forests can provide insights in to which features are important for making predictions, which can be useful for feature selection.

Disadvantages of Random Forests:

1. **Complexity:** They are computationally intensive and may not be as interpretable as individual decision trees.
2. **Memory Consumption:** Since multiple trees are trained, the model can be memory-heavy, especially with large datasets.
3. **Slower Predictions:** The need to aggregate predictions from multiple trees can make prediction time slower compared to simpler models.

Introduction to the Bayes Classifier

The Bayes Classifier is a probabilistic machine learning algorithm that applies Bayes' Theorem to classify data based on prior knowledge (probabilities) and observed data. The idea is to calculate the probability of a class label given the input data, and then choose the class with the highest probability. This approach is particularly useful for classification tasks where the relationships between features and class labels are uncertain or noisy.

Bayes' Theorem:

Bayes' Theorem describes how to update the probability of a hypothesis based on new evidence. It is expressed as:

Steps in the Bayes Classifier:

1. **Prior Probability:** We start with an initial assumption of the likelihood of different classes (before seeing the data).
2. **Likelihood:** For each class, we calculate the probability of observing the given features.
3. **Posterior Probability:** After combining prior and likelihood, we compute the posterior probability for each class.
4. **Prediction:** The class with the highest posterior probability is chosen as the predicted class.

Example: Classifying Fruits

Suppose we want to classify fruits as either an Apple or an Orange based on two features:

- Color (Red or Orange)

- Size (Big or Small)

Step 1: Define the Data

|Fruit | Color| Size |

| | | |

|Apple| Red | Big |

|Apple| Red |Small|

|Apple| Red | Big |

|Orange| Orange |Big |

|Orange| Orange| Small|

|Orange| Orange|Big |

Step 2: Calculate Prior Probabilities

$\{Apple\}=3/6=0.5$

$\{Orange\}=3/6=0.5$

Step 3: Calculate Likelihoods

Let's say we want to classify a fruit with the following features: Color=Red and Size=Small.

- For Apple:

$\{Red\}\{Apple\}=2/3$ (2 apples are red out of 3 apples)

$\{Small\}\{Apple\}=1/3$ (1 small apple out of 3 apples)

- For Orange:

$\{Red\}\{Orange\}=0$ (no red oranges)

Bayes' Rule and Inference

Bayes' rule is a fundamental theorem in probability that describes how to update our beliefs in light of new evidence. It is mathematically expressed as:

$P(A|B)=P(B|A)P(A)/P(B)$ where:

- $P(A|B)$ is the posterior probability (updated belief after seeing evidence).
- $P(B|A)$ is the likelihood (how well evidence BB supports AA).
- $P(A)$ is the prior probability (initial belief before seeing evidence).

- $P(B)$ is the marginal probability (total probability of evidence BB occurring).

Example: Medical Test for a Disease

Suppose a certain disease affects 1% of a population ($P(D)=0.01$), and there is a diagnostic test with:

- 90% sensitivity ($P(T|D)=0.9$)—it correctly identifies diseased individuals.
- 5% false positive rate ($P(T|\neg D)=0.05$)—it incorrectly flags healthy people.

If a person tests positive (TT), what is the probability they actually have the disease ($P(D|T)$)?

Using Bayes' rule:

So, despite testing positive, the probability of actually having the disease is only 15.4% due to the false positive rate.

Multiclass Classification in Machine Learning

Multiclass classification is a type of classification problem where the model assigns an input instance to one of three or more possible classes. Unlike binary classification (which involves only two classes), multiclass classification deals with multiple categories.

Common Algorithms for Multiclass Classification

1. **Logistic Regression (One-vs.-Rest-Ov R)**
2. **Decision Trees and Random Forests**
3. **Support Vector Machines (One-vs.-One-Ov O)**
4. **Neural Networks (Deep Learning models)**
5. **Naïve Bayes Classifier**

Example: Handwritten Digit Recognition (MNIST Dataset)

One famous example of multiclass classification is recognizing handwritten digits (0-9) from the MNIST dataset. Each image represents a number (0-9), and the model needs to classify each image into one of these ten classes.

Steps to Solve a Multiclass Classification Problem

1. **Data Collection** – Gather labeled data where each instance belongs to one of the classes.
2. **Feature Extraction**—Convert raw data (like images or text) into numerical features.
3. **Model Selection**—Choose an appropriate classification algorithm (e. g., neural networks, SVM, or random forest).
4. **Training**—Train the model using labeled examples.
5. **Evaluation** – Measure performance using metrics like accuracy, precision, recall, and F 1-score.

Naïve Bayes Classifier (NBC)

The **Naïve Bayes classifier** is a probabilistic machine learning algorithm based on **Bayes' theorem**. It assumes that features are **conditionally independent**, which simplifies computations and makes it efficient for classification tasks.

Formula

Using Bayes' theorem:

$$P(C|X) = P(X|C) P(C) / P(X)$$

where:

- $P(C|X)$ = Posterior probability (probability of class CCC given data XXX)
- $P(X|C)$ = Likelihood (probability of data XXX given class CCC)
- $P(C)$ = Prior probability (probability of class CCC before seeing data)
- $P(X)$ = Marginal probability of data XXX

Types of Naïve Bayes Classifiers

1. **Gaussian Naïve Bayes** – For continuous data, assumes a normal distribution.
2. **Multinomial Naïve Bayes** – Used for text classification (e. g., spam filtering).
3. **Bernoulli Naïve Bayes** – Deals with binary features (e. g., word presence in text).

Example: Spam Email Classification

We classify emails as Spam (S) or Not Spam ($\neg S$) based on words in the email.

- **Prior probabilities:**
 - $P(S) = 0.3$ (30% of emails are spam)
 - $P(\neg S) = 0.7$ (70% are not spam)
- **Likelihood (probability of words given spam/not spam):**
 - $P(\text{Free} | S) = 0.8$, $P(\text{Free} | \neg S) = 0.1$

If an email contains the word—Free, the probability that it is spam is: P

$$P(S | \text{Free}) = P(\text{Free} | S) P(S) / P(\text{Free})$$

This calculation helps determine if an email should be classified as spam or not.

UNIT III

1. What is a Decision Tree in Machine Learning and how it works?
2. Elaborate Measures and Properties used in Decision Trees for Classification.
3. Elaborate Decision Trees for Regression.
4. What is Bias-Variance Trade-off?
5. Elaborate Random Forests for Classification and Regression.
6. Elaborate Introduction to the Bayes Classifier.
7. Elaborate Bayes' Rule and Inference.
8. Elaborate Multiclass Classification in Machine Learning.
9. Explain Naïve Bayes Classifier (NBC).

UNIT-IV

1. Introduction to Linear Discriminants

Linear Discriminants are a fundamental concept in **supervised machine learning** and are primarily used for **classification tasks**. They are particularly useful when we have multiple classes, and we want to **find a linear combination of features** that best separates these classes.

The most common algorithm based on linear discriminants is Linear Discriminant Analysis (LDA), which projects high-dimensional data onto a lower-dimensional space

While preserving the class separability. This makes LDA not only a classification tool But also a dimensionality reduction technique.

Key Concepts of Linear Discriminants:

1. **Linearity:** Linear Discriminants aim to find a linear boundary that separates the different classes in the data. The "discriminants" are the linear combinations of features that help distinguish between these classes.
2. **Maximizing Between-Class Variance:** LDA tries to maximize the variance between different classes (class separability) while minimizing the variance within each class (compactness of each class).
3. **Dimensionality Reduction:** In LDA, the dimensionality is reduced by projecting the data points onto a **lower-dimensional subspace** where the classes are as separable as possible.

Linear Discriminant Analysis (LDA) Overview:

LDA works by computing the **linear combinations** of the features that best separate the classes. It does so by looking at:

- **Within-class scatter matrix:** Measures the variance within each class.
- **Between-class scatter matrix:** Measures the variance between the different classes.

LDA then finds a projection where the ratio of the **determinant of the between-class scatter matrix** to the **determinant of the within-class scatter matrix** is maximized.

Steps of LDA:

1. **Compute the Mean Vectors:**
 - Compute the mean of each class and the overall mean of the dataset.
2. **Compute Scatter Matrices:**
 - **Within-class scatter matrix (SW):** Measures the spread of data points within each class.
 - **Between-class scatter matrix (SB):** Measures the spread of class means relative to the overall mean.
3. **Compute the Eigen values and Eigenvectors:**
 - The goal is to maximize the separation between classes, so we calculate the Eigen vectors and eigenvalues of the matrix.
 - The top **k Eigen vectors** (those corresponding to the largest eigenvalues) are chosen to form a new space.

4. Projection:

- Project the data points on to the new sub space formed by the top eigenvectors.

5. Classification:

- Use a classification algorithm (like K-Nearest Neighbors (KNN) or **Logistic Regression**) on the transformed data.

Example: Classifying Students' Scores with LDA

Suppose a school wants to classify students based on their Exam Scores and Study Hours as either Pass or Fail.

Student Data:**Student Study Hours Exam Score Class (Pass/Fail)**

A	2	50	Fail
B	3	55	Fail
C	5	70	Pass
D	7	80	Pass
E	8	90	Pass

Step 1: Compute the Mean Vectors

- Mean of Pass: Mean (Study Hours)=7.33, Mean (Exam Score)=80
- Mean of Fail: Mean (Study Hours)=2.5, Mean (Exam Score)= 52.5

Step 2: Compute Scatter Matrices

- **Within-Class Scatter Matrix (SW):** Measures how dispersed the students are within each class (Pass and Fail).
- **Between-Class Scatter Matrix (SB):** Measures how far the Pass and Fail Groups are from each other.

Step 3: Compute Eigenvectors

- Eigenvectors corresponding to the largest eigenvalues are selected, reducing the feature space to a lower dimension (e. g., 1 D).

Step 4: Project the Data

- Project the data points (students) on to this new line that maximizes the separation between classes.

Step 5: Classification

- Using the new 1 D projection, classify new data points (students) as Pass or Fail Based on their position relative to the decision boundary.

Advantages of Linear Discriminants in Machine Learning:

- ✓Dimensionality Reduction: LDA reduces the complexity of the data while preserving class separability.
- ✓Improves Classification Performance: By maximizing class separability, LDA can Lead to improved performance compared to simple models like logistic regression.
- ✓Computational Efficiency: LDA is relatively fast because it does not require iterative optimization (unlike other algorithms like SVM).
- ✓Good for Gaussian Data: If the data is normally distributed, LDA tends to work very Well.

Applications of Linear Discriminants:

- **Face Recognition:** Reducing dimensionality of face images while preserving identity.
- **Medical Diagnosis:** Classifying patients based on diagnostic tests into different disease categories.
- **Spam Detection:** Classifying emails as spam or non-spam based on features like word frequency.

When to Use Linear Discriminants:

- When you have **more than two classes** and want to reduce dimensionality while maintaining class separability.
- When the data points for each class are **normally distributed** and you need a **Linear boundary**.

Linear Discriminants for Classification

Linear Discriminants for Classification is a classification method where the goal is to find a decision boundary that can best differentiate between classes using linear combinations of features. This boundary (or hyper plane) is calculated in such a way that the classes are as separated as possible.

The Linear Discriminant Function for a data set is a linear combination of input features, i. e.,:

$$y = w_1 \times x_1 + w_2 \times x_2 + \dots + w_n \times x_n + b$$

Where:

- $x_1, x_2, \dots, x_n$ are the features.
- $w_1, w_2, \dots, w_n$ are the weights or coefficients.
- B is the bias term.

LDA (Linear Discriminant Analysis) is used to learn these weights in such away that the boundary between classes is optimized. LDA assumes that the features of each class follow a Gaussian distribution with the same covariance matrix, and it tries to find the best projection of the data.

Simple Example: Linear Discriminant for Classification**Problem:**

Imagine you have a dataset where you are trying to classify whether a fruit is an **apple** or orange based on two features: weight and color (represented numerically).

Here's a small dataset:

Fruit	Weight (grams)	Color (R for Red, G for Green)	Class
Apple	150	0.9	Apple
Apple	180	0.95	Apple
Apple	200	0.8	Apple
Orange	180	0.5	Orange
Orange	160	0.4	Orange
Orange	170	0.45	Orange

Steps for Linear Discriminant:

1. **Visualizing the Data:** You plot the data points on a 2 D plane with weight on the x-axis and color on the y-axis. Apples are plotted in one color, and oranges in another.
2. **Finding the Linear Decision Boundary:**
 - You want to find a line (or decision boundary) that separates apples from oranges in the weight and color feature space.
 - LDA calculates the line that maximizes the separation between apples and oranges by considering both the mean and the variance of each class.

The decision boundary is the line that best divides the space such that:

- Apples are mostly on one side.
- Oranges are mostly on the other side.

3. **Classification Rule: Once the boundary is found, it can be used to classify new data points. For example:**

- If a new fruit has a weight of 175 grams and a color value of 0.6 (e. g., it's a fruit that is partially ripe), the linear discriminant function will determine which side of the line it lies on — the apple side or the orange side.

Result:

After applying Linear Discriminants, you'll have a linear decision boundary like this:

- Fruits with **higher color values** and **heavier weights** are more likely to be apples.
- Fruits with **lower color values** and **lighter weights** are more likely to be oranges.

For a new fruit with unknown class, the decision boundary will predict whether it's an apple or an orange based on its weight and color.

Perceptron Classifier: Overview

The Perceptron is one of the simplest types of artificial neural networks used for binary classification tasks. It's a linear classifier that makes predictions based on a weighted sum of the input features. The perceptron algorithm is particularly useful when you have linearly separable data, meaning that the data can be perfectly separated into two classes by a straight line (in two dimensions) or a hyper plane (in higher dimensions). The perceptron uses an activation function to decide which class the input belongs to. The most commonly used activation function is the **step function** (or threshold function), which outputs a binary value (0 or 1, or -1 and 1 depending on the implementation).

The Perceptron model consists of:

1. **Input features ($x_1, x_2, \dots, x_n$):** These are the features of the data points you're trying to classify.
2. **Weights ($w_1, w_2, \dots, w_n$):** These are the parameters of the model that are learned during the training process.
3. **Bias (b):** A constant term added to the weighted sum, which allows the

decision boundary to be shifted.

4. **Activation function: The step function or threshold function that determines the class.**

Perceptron Learning Algorithm

The Perceptron Learning Algorithm is a fundamental method for training a single- layer perceptron. It works by iteratively adjusting the model's weights until it correctly classifies all training examples or reaches a maximum number of iterations.

Algorithm Steps:

1. **Initialize Weights and Bias:**
 - Start with small random values for weights (w) and bias (b).
2. **Loop Through the Data:**
 - For each data point x :
 1. Compute the weighted sum: $y_{\text{pred}} = \text{sign}(w \cdot x + b)$
 2. If $y_{\text{pred}} \neq y_{\text{true}}$ update weights:
 $w = w + \eta(y_{\text{true}} - y_{\text{pred}}) x$
 $b = b + \eta(y_{\text{true}} - y_{\text{pred}})$

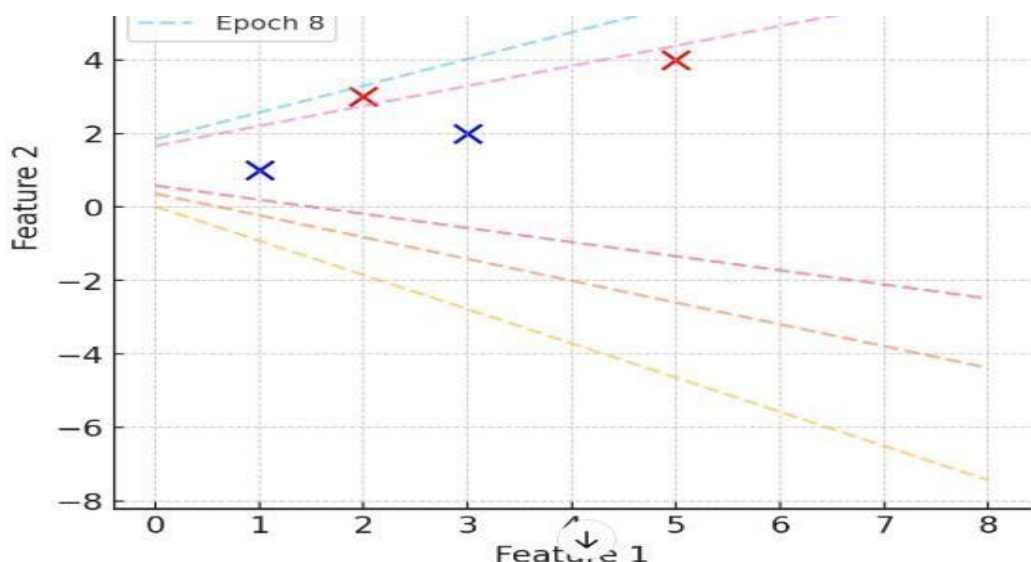
Repeat for multiple **epochs** until convergence.

Simple Example: Manually Working Through Perceptron Algorithm

Let's consider a small dataset:

Feature 1 (x 1)	Feature 2 (x 2)	Class Label (y)
2	3	+1
1	1	-1
5	4	+1
6	6	+1
3	2	-1
7	8	+1

We'll visualize how the decision boundary changes over training.



Here's an updated visualization of the Perceptron Learning Algorithm:

- **Red and blue points** represent two different classes.
- **Dashed lines** show how the decision boundary evolves over multiple epochs.
- Overtime, the perceptron learns to correctly classify all points.

Support Vector Machines (SVM)-Brief Overview

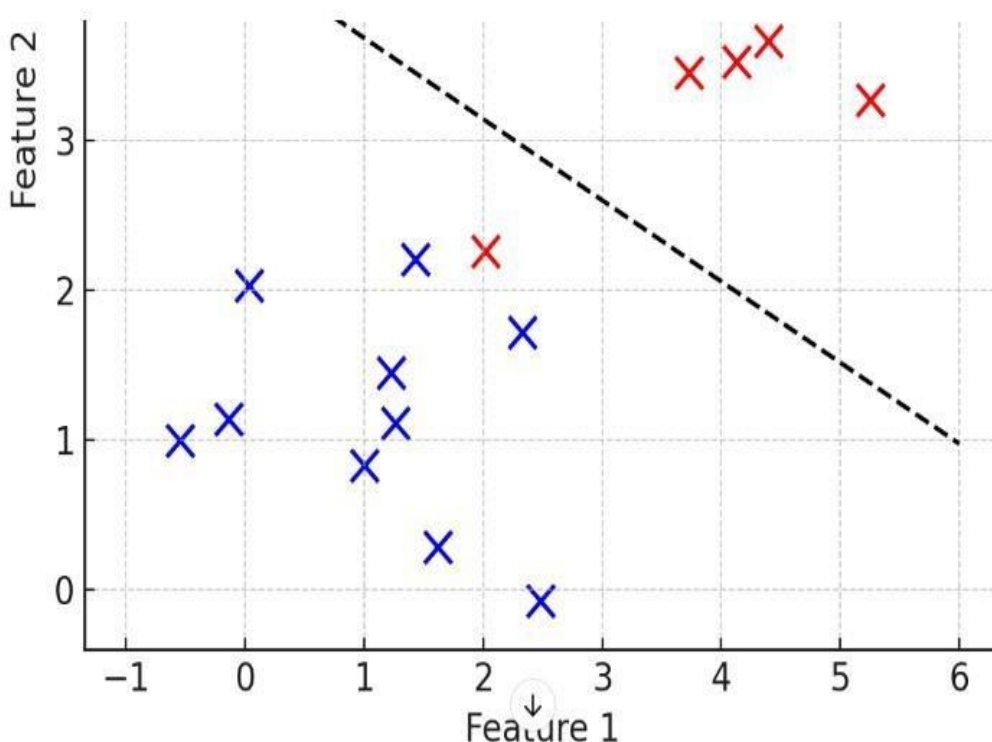
A Support Vector Machine (SVM) is a supervised learning algorithm used for classification and regression tasks. It aims to find the optimal hyper plane that best separates different classes in a dataset.

Key Concepts:

1. **Hyper plane:** A decision boundary that separates classes.
2. **Margin:** The distance between the hyper plane and the closest points from each class.
3. **Support Vectors:** The data points closest to the hyper plane that define the margin.
4. **Soft Margin SVM:** Allows misclassification in case of overlapping data.
5. **Kernel Trick:** Transforms data into a higher dimension to handle non-linear separation.

Example: SVM for Binary Classification

Let's visualize an SVM model that classifies two sets of points.



Visualization of SVM Classification

- **Red and blue points** represent two different classes.
- The dashed black line is the decision boundary (optimal hyper plane).
- The circled points are the support vectors, which define the margin.

Non-Linear SVM with Kernel Trick

When data is not linearly separable, SVM uses the kernel trick to transform it into a higher-dimensional space where a linear separation is possible.

Common Kernel Functions:

1. **Polynomial Kernel:**

$$K(x, x') = (x \cdot x' + c)^d$$

- Useful when data has curved boundaries.

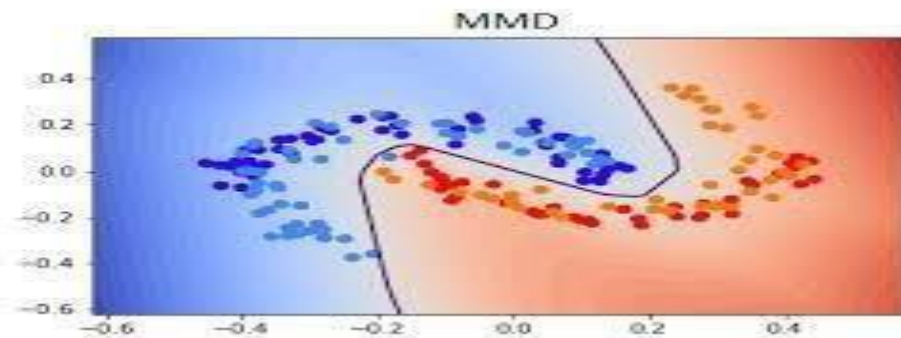
2. Radial Basis Function (RBF) Kernel:

$$K(x, x') = \exp \left(-\gamma \|x - x'\|^2 \right)$$

- Handles highly non-linear patterns.

Example: SVM with RBF Kernel

Let's visualize an SVM classifier on a dataset with non-linearly separable classes.



Visualization of Non-Linear SVM with RBF Kernel

- **Red and blue points** represent two different classes.
- The **curved black decision boundary** separates the classes using the **RBF kernel**.
- The background color gradient shows the SVM confidence regions.

The RBF kernel effectively maps the data to a higher-dimensional space, allowing non-linear classification.

Logistic Regression

Logistic Regression is a supervised learning algorithm used for binary classification. Instead of fitting straight line like linear regression, it models the probability that a given input belongs to a certain class.

Mathematical Formula

Logistic Regression uses the sigmoid function to transform linear outputs into probabilities:

$$P(y=1 | X) = \frac{1}{1 + e^{-(w \cdot X + b)}}$$

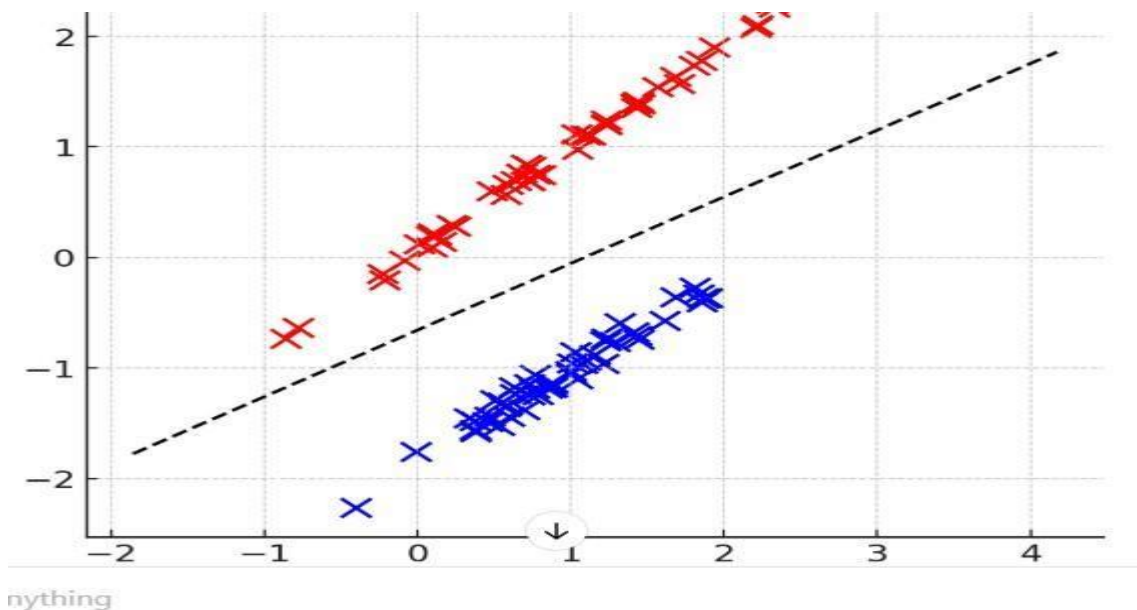
- w =weights
- X =input features
- b =bias
- e =Euler's number

The model predicts class 1 if the probability is greater than 0.5, otherwise, it predicts class 0.

Example: Logistic Regression on a Simple Dataset

Let's visualize how logistic regression classifies two sets of points.

Logistic Regression Visualization



- **Red and blue points** represent two different classes.
- The dashed black line is the decision boundary, separating the classes.
- Logistic regression uses the **sigmoid function** to estimate probabilities and classify data.

Linear Regression

What is Linear Regression?

Linear Regression is a supervised learning algorithm used for predicting continuous values. It models the relationship between a dependent variable (target) and one or more independent variables (features) by fitting a straight line.

Types of Linear Regression

1. **Simple Linear Regression:**
 - One independent variable (X) is used to predict the dependent variable (Y).
 - Example: Predicting house prices based on size.
2. **Multiple Linear Regression:**
 - Multiple independent variables (X₁, X₂, X₃,...) are used to predict Y.
 - Example: Predicting house prices based on size, number of rooms, and location.

Mathematical Representation

1. Simple Linear Regression

For a single independent variable x, the equation is: y

$$= w x + b$$

where:

- y=predicted value
- x=input feature
- w=weight (slope of the line)
- b=bias (intercept)

The goal is to find w and b such that the line **minimizes the error**.

2. Multiple Linear Regression

For multiple independent

variables: $y = w_1 x_1 + w_2 x_2 + \dots + w_n x_n + b$

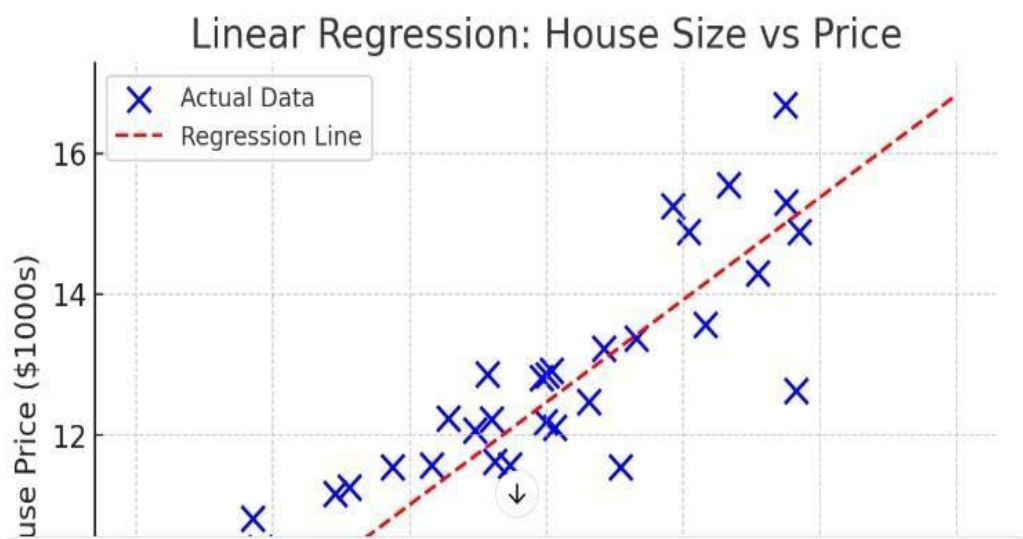
2+...+w_nx_n+b

Where each x_i represents a feature.

Example: Predicting House Prices Based on Size

Let's train a Simple Linear Regression model.

Linear Regression Visualization



- **Blue points** represent real house size and price data.
- The **red dashed line** is the **best-fit regression line**.
- The model predicts house prices based on size.

Multi-Layer Perceptrons (MLPs)

what is an MLP?

A **Multi-Layer Perceptron (MLP)** is a type of **artificial neural network (ANN)** that consists of multiple layers of neurons. Unlike a simple perceptron, which can only handle **linearly separable data**, an MLP can learn **non-linear relationships** using **hidden layers and activation functions**.

Structure of an MLP

An MLP consists of the following layers:

1. **Input Layer:**
 - Accepts raw input features (e. g., pixels in an image or numerical data).
 - The number of neurons = the number of input features.
2. **Hidden Layers:**
 - One or more layers between input and output.
 - Each neuron in a hidden layer applies a weighted sum followed by an activation function (e. g., Re LU, sigmoid).
 - Introduces non-linearity, allowing MLPs to learn complex patterns.
3. **Output Layer:**
 - Produces final predictions.
 - For classification: Uses soft max (multi-class) or sigmoid (binary).
 - For regression: Uses a linear activation function.

Brief Notes on Back propagation for Training an MLP what is Back propagation?

Back propagation (backward propagation of errors) is an optimization algorithm used to train Multi-Layer Perceptrons (MLPs) by adjusting weights to minimize error. It works by propagating the error backward from the output layer to the input layer.

Steps in Back propagation

1. **Forward Pass:**
 - Inputs pass through the network layer by layer.
 - Each neuron applies **weights and an activation function** to compute outputs.
2. **Compute Loss (Error):**
 - Loss is measured as the difference between predicted output and actual output.
 - Example: Mean Squared Error (MSE) or Cross-Entropy Loss.
3. **Backward Pass (Gradient Computation):**
3. **Backward Pass (Gradient Computation):**

- Using partial derivatives and the chain rule, gradients are computed for each weight.
- 4. **Update Weights (Gradient Descent):**
 - Weights are updated using Gradient Descent to minimize error:

Simple Example

Consider an MLP with:

- **2 input neurons, 1 hidden layer (3 neurons), and 1 output neuron.**
- **Activation function: Re LU in hidden layers, Sigmoid in output layer.**
- **Loss function: Cross-Entropy Loss for classification.**

Training Process

1. **Forward Propagation: Compute the predicted output.**
2. **Compute Loss: Compare prediction with actual label.**
3. **Backward Propagation: Adjust weights using gradients.**
4. **Repeat for multiple epochs until convergence.**

UNITIV

1. **Elaborate Linear Discriminants and its Key Concepts.**
2. **Elaborate the steps for LDA.**
3. **Explain Linear Discriminants for Classification.**
4. **Elaborate Perceptron Classifier.**
5. **Elaborate Perceptron Learning Algorithm.**
6. **Elaborate Support Vector Machines.**
7. **Elaborate Logistic Regression and Linear Regression.**
8. **Explain Multi-Layer Perceptrons.**
9. **Explain Back propagation for Training.**

UNIT-5

1. Introduction to Clustering

- **Clustering** is an unsupervised machine learning technique used to group similar data points into clusters based on their characteristics.
- Objective: Maximize similarity within clusters and minimize similarity between clusters.
- Applications: Market segmentation, image segmentation, document categorization, anomaly detection.

Natural Grouping-Clustering is subjective

A set of data instances or samples can be grouped differently based on different criteria or features, in other words clustering is subjective. Figure 23.1 shows a set of seven people. They have been grouped into three clusters based on whether they are school employees, they belong to a family or based on the gender. Therefore choosing the attributes or features based on which clustering is to be carried out is an important aspect of clustering just as it was for classification.



Figure 23.1 Grouping of People based on Different

Clusters–Distance viewpoint

When we are given a set of instances or examples represented as a set of points, we need to define the notion of distance between these points. We then group the points into some number of clusters, such that members of a cluster are close or similar to each other while members of different clusters are dissimilar or farther apart than members belonging to the same cluster. Figure 23.2 shows a data set that has three natural clusters where the data points group together based on the distance. An outlier is a data point that is isolated from all other data points.

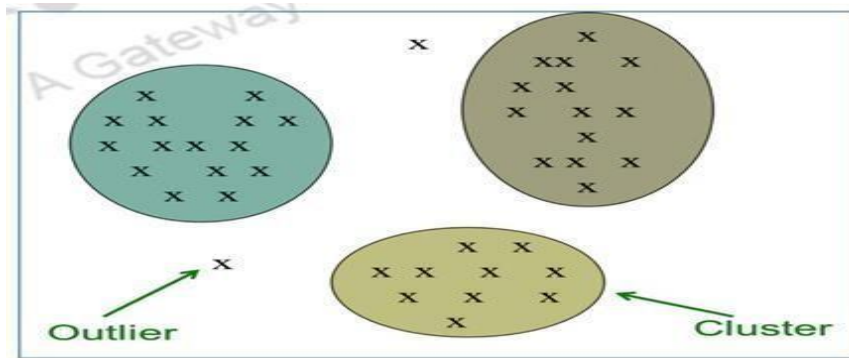


Figure 23.2 Clusters and Outlier

Let us see some real-life examples of clustering. When designing T-shirts making the m to fit each person is too expensive while one-size-fits-all is not a satisfactory policy. We could group people with similar sizes to design—small|| ,—medium || and —large || T-shirts. Example 1: groups people of similar sizes together to make —small|| ,—medium || and—large || T-Shirts.

In today's world of online marketing, segmenting customers according to their similarities would help in targeted marketing. Features such as previous products bought, effect of discounts etc.. Could be used for such marketing.

Another example of clustering is in document clustering where given a collection of text documents, we can cluster them according to their content similarities in order to create a topic hierarchy.

As we can see from the varied applications clustering is one of the most utilized machine learning techniques. In image processing it is used to cluster images based on their visual content. In the web scenario it is used to cluster groups of users based on their access patterns on webpages or cluster searchers based on their search behavior or to cluster webpages based on their content and links. In bioinformatics clustering can be used to group similar proteins based on similarity of their chemical structure and/or functionality. It has been used in almost every field, e. g., medicine, psychology, botany, sociology, biology, archeology, marketing, insurance, libraries, etc. Due to the large increase of online documents text clustering is now becoming very important.

2. Partitioning of Data

- Partitioning involves dividing a dataset into clusters based on similarity or distance metrics.
- Methods: Distance-based (e. g., Euclidean), density-based, or probabilistic.

The methods you mentioned are different approaches in machine learning and data analysis, particularly in clustering, classification, and other algorithms that deal with data structure and relationships. Here's a brief explanation of each:

1. Distance-based Methods (e. g., Euclidean)

- **Euclidean Distance:** This is one of the most commonly used distance metrics. It's used to measure the "straight line" distance between two points in Euclidean space.
 - Formula: $d(p, q) = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2 + \dots}$
 - It's often used in clustering algorithms like **K-means** or **K-nearest neighbors (KNN)**.
 - **Limitations:** It assumes that the data is on the same scale, so it might not work well if features have different units or magnitudes.
- **Manhattan Distance: Another distance metric, also known as L 1 norm, which calculates the sum of absolute differences.**
- **Cosine Similarity:** Often used when the magnitude of the vectors doesn't matter, focusing instead on the angle (useful in text mining and document clustering).

2. Density-based Methods

- These methods focus on grouping data points that are in regions of high density. Unlike distance-based methods that may rely on predefined centroids (like in K-means), density-based methods do not require the number of clusters to be specified beforehand. They can also detect irregular shapes of clusters.
- **DBSCAN (Density-Based Spatial Clustering of Applications with Noise) is a common density-based algorithm. It identifies dense regions of points and separates them from sparse regions.**
 - **Core Idea:** Points within a certain neighborhood (based on distance) are considered part of the same cluster. If a point has enough neighbors (points within a defined radius), it's considered a "core point," and other points within the neighborhood are grouped together.
 - **Advantages:** It can find arbitrarily shaped clusters and handle noise (outliers) well.
- **OPTICS** (Ordering Points to Identify the Clustering Structure) is a more advanced version of DBSCAN, providing a reachability plot to visualize the density structure.

3. Probabilistic Methods

- These methods assume that data is generated by some underlying probabilistic model, and they aim to identify these models from the data. Rather than assigning every point to a cluster with certainty, probabilistic methods compute the probability of a point belonging to a certain cluster.
- **Gaussian Mixture Models (GMMs):** One of the most common probabilistic clustering algorithms. It assumes that the data is a mixture of several Gaussian distributions and assigns probabilities to each data point for belonging to different clusters.
 - **EM Algorithm:** The Expectation-Maximization (EM) algorithm is typically used to estimate the parameters of a GMM, iteratively improving the model by estimating the "soft" cluster assignments of each point.
- **Hidden Markov Models (HMMs):** Often used for sequential data, HMMs are probabilistic models that assume the system being modeled is a Markov process with unobservable (hidden) states.

Each method has its strengths and weaknesses depending on the nature of the data and the problem you're trying to solve!

3. Matrix Factorization

- Matrix factorization decomposes a matrix (e. g., data matrix) into two or more lower-dimensional matrices to reveal hidden patterns or structure.
- Applications: Dimensionality reduction, recommendation systems, latent topic modeling.

Let's break down **matrix factorization** in a simple way with a small example, so it's easy to understand.

Problem Setup:

Imagine you have a matrix where rows represent users and columns represent items (like movies). The entries in the matrix are ratings that users have given to the movies. Some ratings are missing, and our goal is to predict those missing ratings.

Here's a **simple matrix** with user ratings for movies:

User/Movie	Movie 1	Movie 2	Movie 3
User 1	5	?	3
User 2	4	3	?
User 3	?	5	4

In this matrix:

- "?" Means the rating is missing.
- User 1 rated Movie 1 as 5 and Movie 3 as 3, but didn't rate Movie 2.
- User 2 rated Movie 1 as 4, Movie 2 as 3, and didn't rate Movie 3.
- User 3 didn't rate Movie 1, but rated Movie 2 as 5 and Movie 3 as 4.

Matrix Factorization:

Matrix factorization aims to break down this matrix into two smaller matrices, so we can predict the missing ratings.

We want to find two matrices:

1. **User Matrix (U)**: Represents the preferences of users in terms of latent (hidden) factors.
2. **Item Matrix (V)**: Represents the characteristics of the movies in terms of those same latent factors.

The original matrix R (user-item ratings matrix) is approximately equal to the product of these two smaller matrices, U (user features matrix) and V (item features matrix):

$$R \approx U \times V^T$$

Where:

- U has dimensions 3×2 (3 users, 2 latent factors).
- V has dimensions 3×2 (3 movies, 2 latent factors).

Example of the Decomposition:

Let's say we decompose the matrix into two smaller matrices U and V like this:

User Matrix (U):

User	Latent Feature 1	Latent Feature 2
User 1	0.9	0.1
User 2	0.7	0.8
User 3	0.4	0.6

Item Matrix (V):

Movie	Latent Feature 1	Latent Feature 2
Movie 1	1.0	0.5
Movie 2	0.7	1.2
Movie 3	0.3	0.9

Predicted Ratings:

Now, using matrix multiplication, we can approximate the original ratings matrix by multiplying the User Matrix (U) with the Transpose of the Item Matrix (V):

$$R = U \times V^T$$

This will give us predicted ratings for all movies for each user. For example, to predict the rating of User 1 on Movie 2, we compute:

$$R_{1,2} = (0.9 \times 0.7) + (0.1 \times 1.2) = 0.63 + 0.12 = 0.75$$

The Full Predicted Ratings Matrix:

User/Movie	Movie 1	Movie 2	Movie 3
User 1	5	0.75	3
User 2	4	3	3.2
User 3	2.3	5	4

Filling in the Missing Values:

From the predicted ratings matrix, we can see that the missing values are now filled in. For example:

- User 1's rating for Movie 2 was missing, and our predicted value is 0.75 (which could indicate a low rating).
- **User 2's rating for Movie 3 was missing, and we predict it to be 3.2.**

Matrix factorization helps us predict missing values in a matrix by breaking it down into smaller matrices that capture hidden features (latent factors). In our example, we were able to predict the ratings that users would give to movies they hadn't rated yet, based on patterns in the observed ratings.

This technique is widely used in recommendation systems, like Netflix or Amazon, to suggest movies or products to users based on their preferences and similar users' behavior.

4. Clustering of Patterns

- Identifies groups (clusters) in data where patterns within the same cluster are similar, and patterns in different clusters are dissimilar.
- Uses distance metrics like Euclidean, Manhattan, or cosine similarity.

Clustering of patterns refers to the process of grouping similar data points or patterns into clusters or groups in such a way that data points within the same cluster are more similar to each other than to those in other clusters. It is an **unsupervised learning** technique, meaning there are no pre-labeled data points to guide the learning process.

Clustering helps in discovering hidden structures or patterns in datasets, which is particularly useful when you do not have labeled data and want to explore the inherent groupings of the data.

Key Concepts of Clustering

1. **Cluster:** A group of similar data points. Data points in the same cluster should be more similar to each other than to those in different clusters.
2. **Centroid:** A central point that represents a cluster. In some algorithms like K-means, the centroid is the average of the points in the cluster.
3. **Distance Metric:** Measures the similarity or dissimilarity between two data points. Common metrics include:
 - **Euclidean distance:** The straight-line distance between two points in a multi-dimensional space.
 - **Manhattan distance:** The sum of the absolute differences of the coordinates.

Common Clustering Algorithms

1. **K-means Clustering:**
 - One of the most widely used clustering algorithms. It divides the data into K clusters by assigning each data point to the nearest centroid, then recalculating centroids, and repeating this process until convergence.
2. **DBSCAN (Density-Based Spatial Clustering of Applications with Noise):**
 - Groups points that are closely packed together (based on density) and identifies points that lie alone in low-density regions as outliers.

- Unlike K-means, DBSCAN does not require the number of clusters to be specified in advance.
- 3. **Hierarchical Clustering:**
 - Builds a tree-like structure called a **dendrogram**, where each data point starts in its own cluster, and pairs of clusters are merged as we move up the hierarchy.
 - Can be either agglomerative (bottom-up) or divisive (top-down).
- 4. **Gaussian Mixture Models (GMM):**
 - A probabilistic model where each cluster is modeled by a Gaussian distribution. GMM allows for soft clustering, meaning each data point can belong to multiple clusters with different probabilities.

Simple Example of Clustering

Let's break down the concept of **clustering of patterns** with a basic example:

Scenario: Grouping Animals Based on Weight and Height

Imagine we have a data set of animals and their weight and height. We want to cluster them into groups based on these two features to find similar animals.

Here's the dataset:

Animal	Height (cm)	Weight (kg)
Elephant	300	5000
Tiger	100	200
Lion	120	250
Mouse	10	0.02
Giraffe	400	800
Dog	60	20

Step 1: Choose a Clustering Algorithm

Let's use K-means clustering to group the sea animals into 2 clusters based on height and weight (large animals and small animals).

Step 2: K-means Algorithm Process

1. **Initialize Centroids: Randomly pick 2 initial centroids (one for large animals, one for small animals).**
 - Centroid 1: Choose Elephant and Giraffe (large animals).
 - Centroid 2: Choose Mouse and Dog (smaller animals).
2. **Assign Data Points to Nearest Centroid:**
 - Elephant, Giraffe, Tiger, and Lion are assigned to Centroid 1 (large animals).
 - Mouse, Dog are assigned to Centroid 2 (smaller animals).
3. **Update Centroids: Recalculate the centroids as the average of the points in each cluster.**
 - New Centroid 1: Average of Elephant, Giraffe, Tiger, and Lion.
 - New Centroid 2: Average of Mouse and Dog.

4. Repeat: Reassign each animal to the new centroids and recalculate until the centroids no longer change significantly (i. e., convergence).

Step 3: Resulting Clusters

After convergence, we may get the following two clusters:

- **Cluster 1 (Large Animals): Elephant, Giraffe, Tiger, Lion**
- **Cluster 2 (Small Animals): Mouse, Dog**

Clustering of patterns in machine learning involves grouping similar data points into clusters based on features that define those data points. It is widely used in scenarios where we don't have labeled data and need to identify inherent groupings in the data. Clustering algorithms like K-means, DBSCAN, and hierarchical clustering are powerful tools for uncovering patterns in various fields, from marketing to biology.

By grouping similar data points together, clustering allows us to make better sense of large, unstructured datasets and apply machine learning to various real-world problems.

5. Divisive Clustering

- A **top-down hierarchical clustering** approach:
 - Starts with all data in one cluster.
 - Recursively splits clusters until each data point is in its cluster or a stopping criterion is met.
- Computationally intensive but provides a global view.

Divisive Clustering in Machine Learning

Divisive clustering is a **top-down** hierarchical clustering algorithm. It starts with all data points in one large cluster and **recursively divides** it into smaller clusters, following a process of splitting until each data point is in its own cluster (or until a stopping condition is met, such as a predefined number of clusters).

This method contrasts with **agglomerative clustering**, which is a **bottom-up** approach, where each data point starts as its own cluster, and the algorithm merges them progressively into larger clusters.

Key Steps in Divisive Clustering

1. **Start with all data in a single cluster:** All data points are initially in one cluster.
2. **Divide the cluster:** At each step, the algorithm chooses one cluster to split into two smaller clusters. This decision is often based on distance or similarity between data points.
3. **Repeat the process:** Continue dividing the clusters recursively until the stopping criterion is met (e. g., when each data point is in its own cluster or a specified number of clusters is reached).
4. **Form a hierarchical tree (dendrogram):** The resulting divisions can be represented as a tree structure, showing how the data points are divided at each level.

Simple Example of Divisive Clustering

Let's go through a basic example using a small dataset of 6 animals with their **height** and **weight**. We will use divisive clustering to group them into clusters, based on size (height and weight).

Dataset:

Animal Height (cm) Weight (kg)

Elephant	300	5000
Tiger	100	200
Lion	120	250
Mouse	10	0.02
Giraffe	400	800
Dog	60	20

Step 1: Start with One Cluster

Initially, we consider all animals to be a single cluster:

- **Cluster 1:[Elephant, Tiger, Lion, Mouse, Giraffe, Dog]**

Step 2: Divide the Cluster

Now we divide this single large cluster into two smaller clusters. Since the animals differ greatly in size, we can split them into two groups:

- **Large Animals (Elephant, Giraffe)**
- **Small Animals (Tiger, Lion, Mouse, Dog)**

This division is made based on a **simple size criterion**, such as comparing the average size (height and weight) of each animal.

Step 3: Recursively Divide Each Cluster

Next, we continue dividing the clusters:

- **Cluster 1 a (Large Animals):**
 - We divide the **large animals** into two sub-clusters:
 - **Sub-cluster 1 a 1: Elephant (largest animal)**
 - **Sub-cluster 1 a 2: Giraffe (second largest)**
- **Cluster 1 b (Small Animals):**
 - We divide the **small animals** into two sub-clusters:
 - **Sub-cluster 1 b 1: Mouse, Dog (smallest animals)**
 - **Sub-cluster 1 b 2: Tiger, Lion (medium-sized animals)**

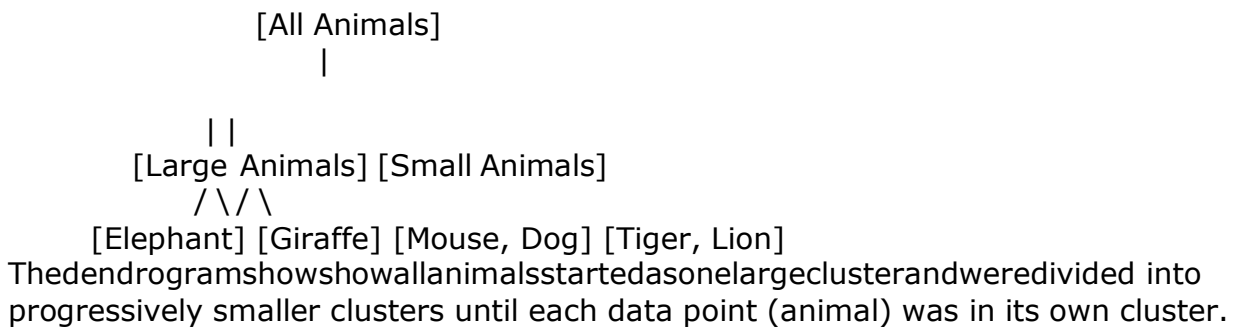
Step 4: Continue Until Each Data Point is a Cluster

Finally, we can continue dividing each cluster until each data point (animal) is in its own cluster:

- **Sub-cluster 1 a 1: Elephant**
- **Sub-cluster 1 a 2: Giraffe**
- **Sub-cluster 1 b 1: Mouse**
- **Sub-cluster 1 b 1: Dog**
- **Sub-cluster 1 b 2: Tiger**
- **Sub-cluster 1 b 2: Lion**

Step 5: Resulting Tree (Dendrogram)

The result of divisive clustering is a **dendrogram**, a tree-like structure that shows how the data points were divided at each step:



Divisive Clustering Algorithm Summary:

1. **Start with a single cluster** containing all data points.
2. **Divide the cluster into smaller groups based on some criterion (e. g., distance, size, or any other measure of similarity).**
3. **Recursively divide** the clusters until each data point is in its own cluster or a stopping condition is met.
4. **Visualize the result as a hierarchical tree (dendrogram).**

Advantages of Divisive Clustering:

- **Hierarchical Structure:** Divisive clustering builds a tree-like structure (dendrogram), which helps visualize the relationships between data points.
- **Top-Down Approach:** Sometimes, a top-down approach is more intuitive and works better when there is a clear division between groups in the dataset.

Disadvantages:

- **Computational Complexity:** Divisive clustering can be computationally expensive, especially for large datasets, as it requires repeatedly splitting clusters and recalculating distances or similarities.
- **Splitting Criteria:** The choice of how to split the clusters (i. e., the splitting criterion) can affect the results and may not always be obvious.

Divisive clustering is a top-down hierarchical clustering algorithm that starts with all data points in one cluster and recursively divides them into smaller groups. It is useful for organizing data into a hierarchy and finding inherent patterns in the data. While it can be computationally intensive, it provides an intuitive way of splitting clusters and visualizing relationships between data points in the form of a dendrogram.

6. Agglomerative Clustering

- A **bottom-up hierarchical clustering** approach:
 - Starts with each data point as a separate cluster.
 - Merges the two closest clusters iteratively based on a link age criterion:
 - **Single Linkage: Closest pair of points.**
 - **Complete Linkage: Farthest pair of points.**
 - **Average Linkage: Mean distance between points.**
 - Stops when all data points form a single cluster or a criterion is met.

Agglomerative Clustering in Machine Learning

Agglomerative Clustering is a type of hierarchical clustering that follows a **Bottom-up approach. It starts by treating each data point as its own cluster and**

Then **iteratively merges** the closest clusters until a stopping condition is met (such as a predefined number of clusters).

It uses a **link age criterion** to determine the distance between clusters:

1. **Single Linkage**–Distance between the closest points of two clusters.
2. **Complete Linkage**–Distance between the farthest points of two clusters.
3. **Average Linkage**–Average distance between all points in two clusters.
4. **Ward's Linkage**–Minimizes variance within clusters.

Example of Agglomerative Clustering

Let's consider five data points with their 1 D values: [2, 10, 15, 40, and 45]

Step-by-Step Process:

1. **Initialization: Each point is its own cluster.**
 - Clusters: {2}, {10}, {15}, {40}, {45}
2. **Step 1 (Merge closest clusters):**
 - Closest points: {10} and {15} (distance=5)
 - New Clusters: {2}, {10, 15}, {40}, {45}
3. **Step 2 (Merge closest clusters):**
 - Closest points: {40} and {45} (distance=5)
 - New Clusters: {2}, {10, 15}, {40, 45}
4. **Step 3 (Merge closest clusters):**
 - Closest points: {10, 15} and {2} (distance=8)
 - New Clusters: {2, 10, 15}, {40, 45}
5. **Step 4 (Final Merge):**
 - Closest clusters: {2, 10, 15} and {40, 45}
 - Final Cluster: {2, 10, 15, 40, 45} (all merged)

Visualizing the Process using a Dendrogram

A **dendrogram** represents the merging process in a tree structure.

```

      {2, 10, 15, 40, 45}
      | |
      | |
      | {40, 45}
      | |
      | |
      | {10, 15}
      | |
      | |
      {2} {10} {15}
  
```

This hierarchical structure helps decide the number of clusters by cutting the dendrogram at a certain height.

Applications of Agglomerative Clustering

- **Customer Segmentation** (e. g., grouping similar customers based on purchasing behavior)
- **Image Segmentation** (e. g., identifying similar regions in an image)
- **Social Network Analysis** (e. g., clustering users based on interests)
- **Bio informatics & Genetics** (e. g., classifying species based on genetic similarity)

7. Partitional Clustering

- Divides the dataset into non-overlapping clusters, with each data point belonging to exactly one cluster.
- Example algorithms: K-Means, K-Medoids.
- Focus is on optimizing a predefined objective function (e. g., minimizing intra-cluster distance).

Partitional Clustering in Machine Learning

What is Partitional Clustering?

Partitional clustering is a technique that divides a dataset into non-overlapping groups (clusters), where each data point belongs to exactly one cluster. Unlike hierarchical clustering (which builds a nested hierarchy of clusters), partitional clustering directly assigns data points into clusters, typically by optimizing a given objective function.

Key Characteristics

- Requires the number of clusters (k) to be predefined.
- Uses an iterative approach to optimize cluster assignments.
- Works well for large data sets compared to hierarchical clustering.
- The most popular algorithm in this category is K-Means.

How Partitional Clustering Works

A general partitional clustering algorithm follows these steps:

1. **Initialize k cluster centroids (randomly or using a specific method).**
2. **Assign** each data point to the nearest cluster centroid.
3. **Recalculate** the centroids based on the assigned points.
4. **Repeat steps 2 and 3 until centroids do not change significantly or a stopping condition is met.**

Example: K-Means Clustering

Let's consider a simple data set with 6 points in a 2 D space:

(1, 1), (2, 1), (4, 3), (5, 4), (8, 7), (9, 8) (1, 1), (2, 1), (4, 3), (5, 4), (8, 7), (9, 8)

Step 1: Choose Number of Clusters (k)

Let's assume we want to divide the data set into $k=2$ clusters.

Step 2: Initialize Cluster Centroids

We randomly pick two points as initial centroids:

- **Centroid 1** → (1, 1) (1, 1)
- **Centroid 2** → (8, 7) (8, 7)

Step 3: Assign Points to the Nearest Centroid

We calculate the Euclidean distance of each point to both centroids:

Point	Distance to (1, 1)	Distance to (8, 7)	Assigned Cluster
(1, 1)	0	9.22	1
(2, 1)	1	8.60	1
(4, 3)	3.60	5.00	1
(5, 4)	5.00	3.61	2
(8, 7)	9.22	0	2
(9, 8)	10.63	1.41	2

Step 4: Compute New Centroids

Each centroid is now updated to the **mean** of its assigned points:

- New Centroid 1 (average of Cluster 1 points): $(1+2+4/3, 1+1+3/3) = (2.33, 1.67)$
- New Centroid 2 (average of Cluster 2 points): $(5+8+9/3, 4+7+8/3) = (7.33, 6.33)$

Step 5: Repeat Steps 3&4 until Convergence

- Reassign points to the new centroids.
- Update centroids again.
- Stop when centroids **no longer change significantly**.

Final Clusters

After a few iterations, we get:

- **Cluster 1:** (1, 1), (2, 1), (4, 3) (1, 1), (2, 1), (4, 3)
- **Cluster 2:** (5, 4), (8, 7), (9, 8) (5, 4), (8, 7), (9, 8)

Advantages & Disadvantages

✓ Advantages

- Fast and efficient (especially for large datasets).
- **Scalable** for big data applications.
- Works well when clusters are **well-separated** and **spherical**.

✓Disadvantages

- Requires specifying k (number of clusters) in advance.
- Sensitive to initialization (bad initial centroids can lead to poor clustering).
- Does not handle **non-spherical** clusters well.
- Sensitive to outliers (they can pull centroids away).

Applications of Partitional Clustering

- **Customer Segmentation** (e. g., grouping similar customers for marketing)
- **Image Compression** (e. g., reducing colors in an image)
- **Anomaly Detection** (e. g., detecting fraud in transactions)
- **Document Clustering** (e. g., grouping news articles by topic)

Partitional clustering is a powerful and widely used technique for clustering large datasets. **K-Means** is the most popular algorithm, but it requires careful selection of k and is sensitive to **initialization** and **outliers**. Despite its limitations, it is widely applied in various domains such as marketing, computer vision, and anomaly detection.

8. K-Means Clustering

- A partitional clustering algorithm that minimizes intra-cluster variance:
 - Initialize k cluster centroids randomly.
 - Assign each data point to the nearest centroid.
 - Update centroids as the mean of points in each cluster.
 - Repeat until centroids stabilize.
- **Limitations:**
 - Sensitive to initialization.
 - Assumes spherical clusters of equal size.

Partitional Clustering in Machine Learning (Brief Explanation)

Partitional Clustering is a clustering technique that divides a dataset into non-overlapping groups (clusters), where each data point belongs to exactly one cluster. It aims to optimize a certain criterion, such as minimizing the intra-cluster distance.

The most popular partitional clustering algorithm is K-Means, which follows these steps:

1. Select k cluster centroids randomly.
2. Assign each data point to the **nearest centroid**.
3. Recalculate the centroids as the **mean** of assigned points.
4. Repeat steps 2&3 until centroids do not change significantly.
5. **Example of K-Means Clustering**
6. Given 6 points in a 2 D space: $(1, 1), (2, 1), (4, 3), (5, 4), (8, 7), (9, 8)$
7. $(1, 1), (2, 1), (4, 3), (5, 4), (8, 7), (9, 8)$
8. **Initialize 2 centroids randomly, e. g., $(1, 1)$ and $(8, 7)$.**

2. Assign each point to the nearest centroid.
 3. Compute new centroids:
 - o Cluster 1 (Closer to (1, 1)): (1, 1), (2, 1), (4, 3) (1, 1), (2, 1), (4, 3) → New Centroid (2.33, 1.67)(2.33, 1.67)
 - o Cluster 2 (Closer to (8, 7)): (5, 4), (8, 7), (9, 8) (5, 4), (8, 7), (9, 8) → New Centroid (7.33, 6.33)(7.33, 6.33)
 4. Repeat until convergence.
- Final Clusters:**
- **Cluster 1:** (1, 1), (2, 1), (4, 3) (1, 1), (2, 1), (4, 3)
 - **Cluster 2:** (5, 4), (8, 7), (9, 8) (5, 4), (8, 7), (9, 8)

Advantages & Disadvantages

- ✓ Fast & scalable
- ✓ Works well when clusters are well-separated
- ✓ requires specifying k in advance
- ✓ **Sensitive to initialization & outliers**

10. Soft Partitioning and Soft Clustering

- **Soft Partitioning allows data points to belong to multiple clusters with membership probabilities.**
- **Soft Clustering methods, such as Fuzzy C-Means, assign degrees of membership rather than hard assignments.**

Soft Partitioning vs. Soft Clustering with Example

Soft partitioning and soft clustering are techniques used when data points do not strictly belong to only one category. Instead of rigid (hard) assignments, they allow overlapping associations with varying degrees of membership.

1. Soft Partitioning

Soft partitioning refers to dividing a dataset into overlapping groups where each item can belong to multiple partitions with a certain degree of membership. This is widely used in **fuzzy logic systems** and **decision-making applications**.

Example: Document Classification

Imagine a document that discusses both "Technology" and "Business." Instead of assigning it to a single category (hard partitioning), soft partitioning allows it to belong partially to both:

- **60%Technology**
- **40%Business**

This helps in applications like **search engines** where documents can be retrieved based on relevance to multiple topics.

2. Soft Clustering

Soft clustering is a clustering method where a data point can belong to multiple clusters with different probability values. It is widely used in **machine learning, image processing, and customer segmentation**.

Example: Customer Segmentation

A company wants to group customers based on shopping behavior. A customer might be categorized as:

- **70% Frequent Buyer**
- **30% Seasonal Shopper**

Instead of strictly classifying the customer into one category, soft clustering acknowledges that customer behavior can overlap across different segments.

Key Difference Illustrated

Feature	Soft Partitioning (Fuzzy Sets)	Soft Clustering (Fuzzy Clustering)
Definition	Overlapping partitions with degrees of association	Clustering where points have probabilistic memberships
Example	A document is 60% Technology, 40% Business	A customer is 70% Frequent Buyer, 30% Seasonal Shopper
Application	Search engines, decision systems	Machine learning, customer analysis

Both techniques are useful when data naturally has overlapping characteristics!

10. Fuzzy C-Means Clustering

Fuzzy C-Means (FCM) Clustering

Fuzzy C-Means (FCM) is a **soft clustering** algorithm where each data point belongs to multiple clusters with varying degrees of membership. Unlike **K-Means**, which assigns each point to exactly one cluster, FCM allows for fuzzy memberships, making it useful for **uncertain or overlapping data**.

Algorithm Steps

1. **Initialize cluster centers (C clusters).**
2. **Assign membership values** for each data point in all clusters. The membership values sum to **1** across clusters.
3. **Update cluster centers** based on weighted averages of points.
4. **Recalculate membership values** using the updated centers.
5. **Repeat until convergence (changes in membership values are minimal).**

Small Example: Clustering 3 Points in to 2

Clusters Dataset (3 points in 2 D space) (1, 2), (2, 3), (8, 8) (1, 2), (2, 3), (8, 8)

Step 1: Initialize 2 cluster centers

Assume random centers:

- **Cluster 1:(1, 1)**
- **Cluster 2:(7, 7)**

Step 2: Compute Membership Values

Using the FCM membership formula, we compute how much each point belongs to each cluster. Example membership values might look like this:

Point Cluster 1 Membership Cluster 2 Membership

(1, 2)	0.9	0.1
(2, 3)	0.8	0.2
(8, 8)	0.2	0.8

Step 3: Update Cluster Centers

New cluster centers are calculated as the weighted mean of points based on their membership.

Step 4: Recalculate Memberships

Using the new centers, update the membership values.

Step 5: Repeat until Convergence

The process continues until membership values stop changing significantly.

Key Features of FCM

- Each point belongs to multiple clusters (soft clustering).
- The degree of membership indicates how closely a point is related to a cluster.
- Used in image segmentation, customer segmentation, and medical diagnosis.

11. Rough Clustering

Rough clustering is a technique based on **Rough Set Theory**, where data points are categorized into three groups:

1. **Lower Approximation** → **Data points that definitely belong to a cluster.**
2. **Upper Approximation** → **Data points that possibly belong to a cluster.**
3. **Boundary Region** → **Data points that have uncertain membership and could belong to multiple clusters.**

Example: Rough Clustering for Student Performance Analysis

A university wants to classify students into two categories: "High Performers" and "**Low Performers**", based on their test scores.

Student Data (Test Scores out of 100):

Student Score

A	90
B	85
C	40
D	35
E	60

Step 1: Define Cluster Membership

- **Lower Approximation (High Performers):** Students who clearly belong in the high-performing group.

- $\{A, B\} \rightarrow$ **Definitely High Performers**
- **Lower Approximation (Low Performers):** Students who clearly belong in the low-performing group.
 - $\{D\} \rightarrow$ **Definitely Low Performer**
- **Boundary Region (Uncertain Students):** Students whose scores are in between and might belong to either group.
 - $\{C, E\} \rightarrow$ **Could be High or Low Performers**

Step 2: Rough Clustering Output

Student Cluster Membership

A, B	Lower Approx.(High Performer)
D	Lower Approx.(Low Performer)
C, E	Boundary Region (Uncertain Category)

Interpretation

- **A & B are definitely High Performers.**
- **D is definitely a Low Performer.**
- **C & E fall in the boundary region (uncertain). They may improve or decline based on future performance.**

Applications of Rough Clustering

- **Medical Diagnosis** → Patients may exhibit symptoms belonging to multiple diseases.
- **Anomaly Detection** → Identifying uncertain cyber security threats.
- **Market Segmentation** → Customers with overlapping purchasing behavior.

12. Rough K-Means Clustering Algorithm

- Extends K-Means by using rough approximations for clusters:
 - Data points are assigned to either the **lower approximation** or the **Boundary**.
 - Reduces ambiguity in clustering while maintaining flexibility.

Rough K-Means Clustering Algorithm

What is Rough K-Means?

Rough K-Means (RKM) is a hybrid clustering algorithm that integrates **K-Means** with **Rough Set Theory** to handle uncertainty in data clustering. Unlike standard K-Means, where each data point is assigned to only one cluster, Rough K-Means introduces the concept of **Lower and Upper Approximations**:

1. **Lower Approximation** → Data points that definitely belong to a cluster.

2. Upper Approximation→ Data points that possibly belong to a cluster (boundary region).

This allows handling noisy, imprecise, or overlapping data more effectively.

Steps of Rough K-Means Algorithm

1. **Initialize K cluster centroids randomly.**
2. **Assign each data point to clusters:**
 - If a data point is **close enough** to only one centroid →Assign it to **Lower Approximation**.
 - If a data point is near multiple centroids → Assign it to Upper Approximation (boundary region).
3. **Update cluster centroids:**
 - Compute centroids using only Lower Approximation data points.
 - The centroids are updated iteratively until convergence.
4. **Repeat steps 2 and 3 until clusters stabilize.**

Example: Rough K-Means for Customer Segmentation

A company wants to classify customers into "Frequent Buyers" and "Occasional Shoppers" based on their purchase frequency.

Customer Data (Purchases in last 6 months):

Customer No. of Purchases

A	50
B	48
C	20
D	15
E	30

Step 1: Initialize K=2 Clusters

- Cluster 1: Frequent Buyers
- Cluster 2: Occasional Shoppers

Step 2: Assign Points to Lower & Upper Approximation

Customer Cluster 1 (Frequent)		Cluster 2 (Occasional)
A, B	Lower Approx.(Definitely Frequent)	✓
D	✓	Lower Approx.(Definitely Occasional)
C, E	Upper Approx.(Boundary)	Upper Approx.(Boundary)

- **A & B are definitely Frequent Buyers.**

- **D is definitely an Occasional Shopper.**
- **C & E are uncertain and belong to both groups (boundary region).**

Advantages of Rough K-Means

- ✓ handles noisy and ambiguous data better than traditional K-Means.
- ✓ Provides a **clear distinction** between **definite** and **uncertain** cluster membership.
- ✓ Useful in medical diagnosis, customer segmentation, and anomaly detection.

13. Expectation-Maximization (EM)-Based Clustering

Expectation-Maximization (EM)-Based Clustering What

is EM-Based Clustering?

Expectation-Maximization (EM) is a **soft clustering** algorithm used for **probabilistic clustering**. It assumes that the data is generated from a mixture of several probability distributions (typically **Gaussian Mixture Models - GMM**). Instead of assigning each point strictly to one cluster (like **K-Means**), EM assigns probabilities that each point belongs to a particular cluster.

How EM Works? (Algorithm Steps)

1. **Initialize: Choose initial cluster parameters (means, variances, and mixing coefficients for Gaussian distributions).**
2. **E-Step (Expectation Step): Compute the probability that each data point belongs to each cluster.**
3. **M-Step (Maximization Step): Update the cluster parameters (means, variances, and probabilities) based on weighted points.**
4. **Repeat Steps 2&3 until convergence (i. e., parameters stop changing significantly).**
5. **Example: EM-Based Clustering on Customer Spending Data**
6. Suppose we have a data set of customers based on their Annual Income (\$1000 s) and
7. **Spending Score (1-100).**
8. **Customer Data (Income, Spending Score):**

Customer	Annual Income (\$1000 s)	Spending Score
A	15	85
B	16	80
C	25	40
D	50	20

Customer	Annual Income (\$1000s)	Spending Score
E	60	10

Step 1: Initialize Gaussian Parameters

- Assume 2 clusters:
 - Cluster 1 (Low Income-High Spending)**
 - Cluster 2 (High Income-Low Spending)**

Step 2: E-Step (Compute Probabilities)

For each customer, compute the probability of belonging to each cluster using the **Gaussian Probability Density Function (PDF)**. **Example results:**

Customer	Probability of Cluster 1	Probability of Cluster 2
A	0.90	0.10
B	0.85	0.15
C	0.60	0.40
D	0.30	0.70
E	0.10	0.90

Step 3: M-Step (Update Parameters)

- Update **mean & variance** of each cluster using weighted averages.
- Update mixing coefficients (proportion of points in each cluster).

Step 4: Repeat until Convergence

The probabilities are updated iteratively until clusters stabilize.

Advantages of EM-Based Clustering

- ✓ handles complex distributions (e. g., overlapping clusters).
- ✓ Probabilistic assignments allow better uncertainty handling.
- ✓ more flexible than K-Means (works with non-spherical clusters).

14. Spectral Clustering

Spectral Clustering: A Simple Example

What is Spectral Clustering?

Spectral Clustering is a graph-based clustering algorithm that uses the eigenvalues of a similarity matrix to perform clustering. It is especially useful for non-linearly separable data, where traditional methods like K-Means fail.

Step-by-Step Example

Scenario:

We have the following 6 data points, and we want to cluster them into two groups:

Point X Y

A	1	2
B	2	3
C	3	2
D	8	8
E	9	7
F	10	8

If we use K-Means, it may fail because the data points could be arranged in a non-linear pattern. Spectral Clustering can handle such situations.

Step 1: Construct a Similarity Matrix

We use a Gaussian similarity function to compute pair wise similarities: Formula

in Simple Form:

For simplicity, assume that A, B, C form one group and D, E, F form another.

	A	B	C	D	E	F
A	1	0.9	0.8	0	0	0
B	0.9	1	0.85	0	0	0
C	0.8	0.85	1	0	0	0
D	0	0	0	1	0.9	0.85
E	0	0	0	0.9	1	0.95
F	0	0	0	0.85	0.95	1

Step 2: Compute Graph Laplacian

- Compute Degree Matrix (D) where diagonal elements are the sum of row values.
- Compute Laplacian Matrix ($L=D-S$).

Step 3: Compute Eigen vectors of the Laplacian Matrix

- Select the **smallest k eigenvectors** to represent the data in a lower-dimensional space.
- Here, we choose $k=2$ (since we want 2 clusters).

Step 4: Apply K-Means on Eigenvectors

- Perform K-Means on the transformed data.
- The algorithm correctly separates the two groups :{ A, B, C} and {D, E, F}.

Final Clusters:

Cluster 1 (Blue) Cluster 2 (Red)

A (1, 2)	D (8, 8)
B (2, 3)	E (9, 7)
C (3, 2)	F (10, 8)

Why Use Spectral Clustering?

- **Handles non-linearly separable data (e. g., circles, and moons).**
- **Graph-based approach works well with complex relationships.**
- **More flexible than K-Means, which assumes spherical clusters.**

UNIT V

1. Elaborate what is Clustering.
2. Elaborate Partitioning of Data.
3. What is Matrix Factorization? Explain.
4. Elaborate in brief, Common Clustering Algorithms.
5. Elaborate the following types of Clustering:
 1. Divisive Clustering
 2. Agglomerative Clustering
 3. K-means Clustering
 4. Fuzzy C-means Clustering
6. Explain 1. Rough Clustering 2. Spectral Clustering
7. Explain Expectation Maximization-Based Clustering.